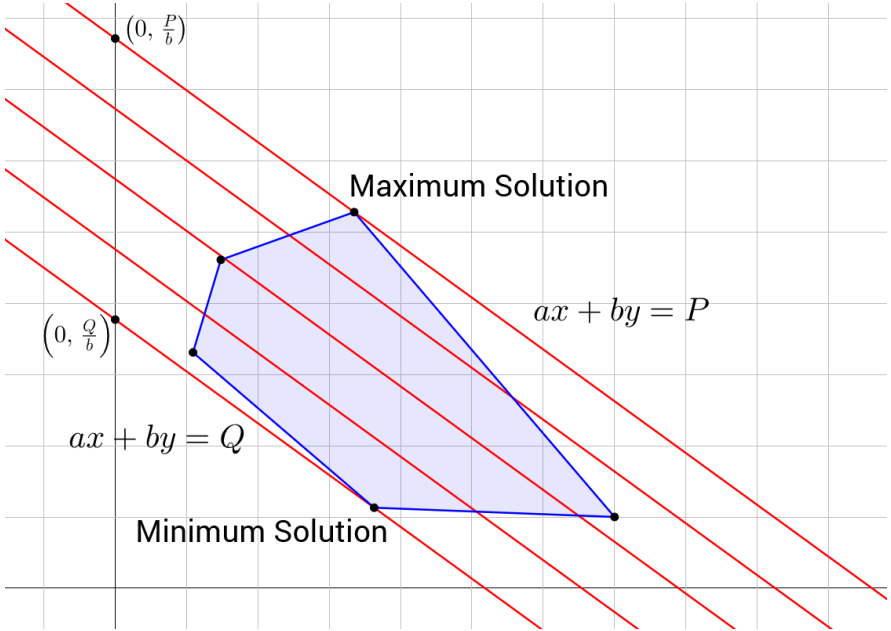
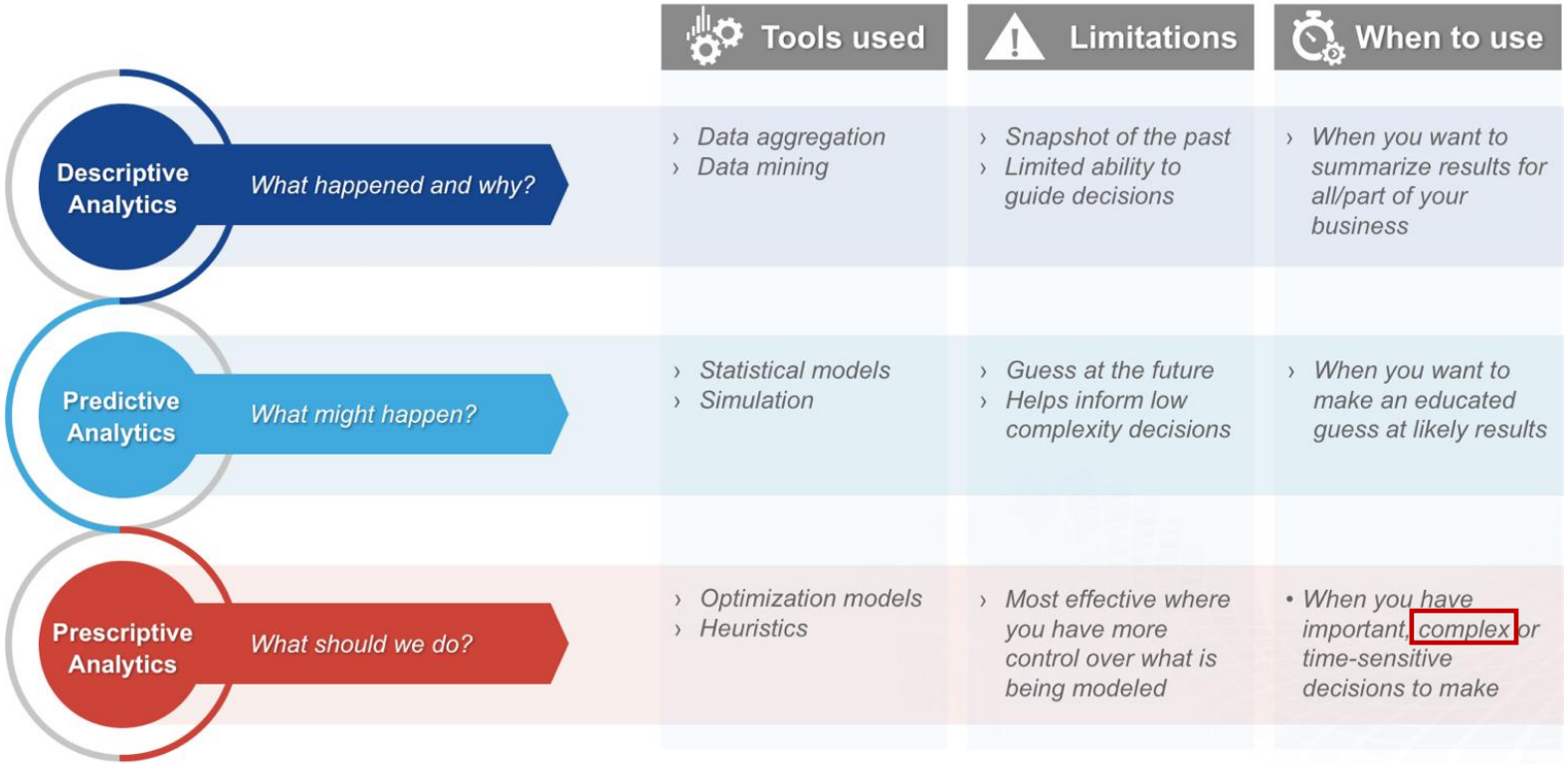
Prescriptive Analytics - Optimization I: Linear Programming



Gerry Tsoukalas

***note**: some case study content courtesy of the Wharton School, UPenn

Recall… the different types of Analytics



What are “complex” actions/decisions?

- Recall some of our labs… these were “simple”
  - **k-NN:** do we target a certain customer for a loan? Yes or No?
  - **MNB:** do we classify a message as spam? Yes or No?
  - **Log. Reg.:** identify people who are likely to complete a task on time
  - **Neural Nets:** predict car sale prices

- In many business settings, decisions are “complex”
  - Complexity in decisions usually comes from desires & constraints
    - k-NN loans: what if we have a limited budget? multiple products to offer? Different loan sizes for each customer?
  - Warning: Complexity is sometimes hidden in seemingly “simple” settings…

How do we deal with complex decisions?

- Two main toolkits
  - #1: Optimization
    - Linear Programming << today's lecture
    - Integer Programming
    - Non-linear Programming
    - Other (more advanced)…
  - #2 Monte Carlo simulation
    - used to simulate future possible scenarios in the future, when there is uncertainty

Linear Programming - Domaine Paul Autard (DPA)

- DPA has 12 hectares of vineyards in Chateauneuf du Pape
  - Each hectare produces about 25 hectoliters of wine
  - That's about 3,400 cases of wine in total
- The vineyards are planted with 3 types of grape
  - 1,700 cases of Grenache
  - 1,200 cases of Syrah
  - 500 cases of Counoise
- The domaine produces two varieties of red wine
  - La Côte Ronde (C). Each case of C requires
    - 0.5 cases of Grenache, 0.5 cases of Syrah
    - will wholesale for 400€ per case
  - Domaine Paul Autard (P). Each case of P requires
    - 0.6 cases of Grenache, 0.2 cases of Syrah, 0.2 cases of Counoise
    - will wholesale for 600€ per case
- How many cases of each wine should Paul Autard make?



Manual solution – the "greedy" algorithm

- **Idea:** Maximize revenues by exhausting sales of the higher-priced wine (P) first
- **Note:** Only Domaine Paul Autard (P) uses counoise grapes
  - One case of P needs 0.2 cases of counoise…
  - …we can therefore make a maximum of 500/0.2 = 2500 cases of P
- Verify we have enough Syrah and Grenache grapes to do this
  - Grenache needed: 0.6*2500 = 1500 cases (out of 1700)
  - Syrah needed: 0.2*2500 = 500 cases (out of 1200)
- What is left for product of Côte Ronde (C):
  - 200 cases of Grenache
  - 700 cases of Syrah
  - The 50/50 ratio means we will run out of Grenache first
  - We can thus make 200/0.5 = 400 cases of C
- Final answer:
  - Make 400 cases of C
  - Make 2500 cases of P



Two main steps of optimization

- Step 1: Re-express the verbal description of the problem in a mathematical/algebraic form
  - Algebraic formulation needs to follow universal standards (tbd)
- Step 2: Use software (e.g., python add-in, or excel solver) to solve the “model”

Step 1: Linear Programming (LP) notation

- given the ***decision variables****:*

1) Decision Variables

- maximize the ***objective function value*** (in thousands of Euros)

2) Objective Function

- subject to ***the constraints***:

3) Constraints

Step 1: Linear Programming (LP) notation

- given the ***decision variables****:*

  C = cases of la Côte Ronde to produce

  P = cases of Domaine Paul Autard to produce

- maximize the ***objective function value*** (in thousands of Euros)

2) Objective Function

- subject to ***the constraints***:

3) Constraints

Step 1: Linear Programming (LP) notation

- given the ***decision variables****:*

  C = cases of la Côte Ronde to produce

  P = cases of Domaine Paul Autard to produce

- maximize the ***objective function value***

  OFV = 400 C + 600 P

- subject to ***the constraints***:

3) Constraints

Step 1: Linear Programming (LP) notation

- given the ***decision variables***:

  C = cases of la Côte Ronde to produce

  P = cases of Domaine Paul Autard to produce

- maximize the ***objective function value***

  OFV = 400 C + 600 P

- subject to ***the constraints***:

  $0.5\,C + 0.6\,P \leq 1700$ (Grenache)

  $0.5\,C + 0.2\,P \leq 1200$ (Syrah)

  $0.2\,P \leq 500$ (Counoise)

  $1\,C \geq 0$ (non-negativity)

  $1\,P \geq 0$ (non-negativity)

Step 1: Linear Programming (LP) notation

- given the ***decision variables***:

  C = cases of la Côte Ronde to produce

  P = cases of Domaine Paul Autard to produce

- maximize the ***objective function value***

  OFV = 400 C + 600 P

- subject to ***the constraints***:

  0.5 C + 0.6 P ≤ 1700 (Grenache)

  0.5 C + 0.2 P ≤ 1200 (Syrah)

  0.2 P ≤ 500 (Counoise)

  1 C ≥ 0 (non-negativity)

  1 P ≥ 0 (non-negativity)

Step 1: Linear Programming (LP) notation

- To deal with smaller numbers, let's do a change of variables…

  C = 100's of cases of la Côte Ronde to produce

  P = 100's of cases of Domaine Paul Autard to produce

Step 1: Linear Programming (LP) notation

- To deal with smaller numbers, let's do a change of variables…

  C = 100's of cases of la Côte Ronde to produce

  P = 100's of cases of Domaine Paul Autard to produce

- maximize the ***objective function value ,*** in thousands of Euros

  $OFV = 40\,C + 60\,P$

- subject to ***the constraints***:

$$
\begin{aligned}
0.5\,C + 0.6\,P &\leq 17 \quad \text{(Grenache)} \\
0.5\,C + 0.2\,P &\leq 12 \quad \text{(Syrah)} \\
0.2\,P &\leq 5 \quad \text{(Counoise)} \\
1\,C &\geq 0 \quad \text{(non-negativity)} \\
1\,P &\geq 0 \quad \text{(non-negativity)}
\end{aligned}
$$

Step 2 - Optimization in Python

- Issue: hundreds of different optimization addons/packages…
- Each optimization engine has its own syntax…
- Use a generic modeling package to standardize syntax, then call solution engines as needed
- We will work with:



- http://www.pyomo.org/
  - “Pyomo is a Python-based, open-source optimization modeling language with a diverse set of optimization capabilities.”

PYOMO syntax

- Step 0: Load “Domain Paul Autard” Notebook
- Step 1: Load PYOMO environment

```
# Install Pyomo optimization modeling package
!pip install pyomo
import pyomo.environ as pyo
from pyomo.environ import *
from pyomo.opt import SolverFactory
```

```
Collecting pyomo
  Downloading Pyomo-6.1.2-cp37-cp37m-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_12_x86_64.manylinux2010_x86_64.whl (9.1 MB)
     |████████████████████████████████| 9.1 MB 5.1 MB/s
Collecting ply
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
     |████████████████████████████████| 49 kB 5.1 MB/s
Installing collected packages: ply, pyomo
Successfully installed ply-3.11 pyomo-6.1.2
```

PYOMO syntax

- Step 2: Load Optimization Engine: “GLPK” for linear programs

```
# Install optimization engine (solution algorithms)
# Linear program solver https://www.gnu.org/software/glpk/
!apt-get install -y -qq glpk-utils
```

```
Selecting previously unselected package libsuitesparseconfig5:amd64.
(Reading database ... 155047 files and directories currently installed.)
Preparing to unpack .../libsuitesparseconfig5_1%3a5.1.2-2_amd64.deb ...
Unpacking libsuitesparseconfig5:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libamd2:amd64.
Preparing to unpack .../libamd2_1%3a5.1.2-2_amd64.deb ...
Unpacking libamd2:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libcolamd2:amd64.
Preparing to unpack .../libcolamd2_1%3a5.1.2-2_amd64.deb ...
Unpacking libcolamd2:amd64 (1:5.1.2-2) ...
Selecting previously unselected package libglpk40:amd64.
Preparing to unpack .../libglpk40_4.65-1_amd64.deb ...
Unpacking libglpk40:amd64 (4.65-1) ...
Selecting previously unselected package glpk-utils.
Preparing to unpack .../glpk-utils_4.65-1_amd64.deb ...
Unpacking glpk-utils (4.65-1) ...
Setting up libsuitesparseconfig5:amd64 (1:5.1.2-2) ...
Setting up libcolamd2:amd64 (1:5.1.2-2) ...
Setting up libamd2:amd64 (1:5.1.2-2) ...
Setting up libglpk40:amd64 (4.65-1) ...
Setting up glpk-utils (4.65-1) ...
Processing triggers for libc-bin (2.27-3ubuntu1.3) ...
/sbin/ldconfig.real: /usr/local/lib/python3.7/dist-packages/ideep4py/lib/libmkldnn.so.0 is not a symbolic link

Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
```

PYOMO syntax

- Step 3: Initialize model
  - Two main types of Pyomo models
    - Concrete Models: data is provided
    - Abstract Models: data is replaced by labeled notation, and needs to be loaded subsequently

We will mainly work with Concrete models:

```
[ ]  # Initialize model by defining a pyomo model
     model = pyo.ConcreteModel()
```

PYOMO syntax

- Step 4: Declare decision variables with bounds

```
[ ]  # ------------------------------------
     # Decision Variables                 |
     # ------------------------------------

     # c is the number of cases of Cote Ronde (100's of cases)
     # p is the number of cases of Domaine Paul Autard to produce (100's of cases)
     model.c = pyo.Var(bounds=(0, None))
     model.p = pyo.Var(bounds=(0, None))

     # Shortcut: redefine pyomo variable names to use shorthand
     c = model.c
     p = model.p
```

PYOMO syntax

- Step 5: Declare Objective Function

```
# ------------------------------------
# Objective Function
# ------------------------------------

# Objective function definition
model.obj = pyo.Objective(expr = 40*c + 60*p, sense = maximize)
```

PYOMO syntax

- Step 6: Declare Constraints

```
# ------------------------------------
# CONSTRAINTS
# ------------------------------------

# Constraint definition, 3 constraints in total named C1, C2, C3
model.C1 = pyo.Constraint(expr = 0.5*c + 0.6*p <= 17, doc = 'Grenache Constraint')
model.C2 = pyo.Constraint(expr = 0.5*c + 0.2*p <= 12, doc = 'Syrah Constraint')
model.C3 = pyo.Constraint(expr = 0.0*c+0.2*p <= 5, doc = 'Counoise Constraint')
```

PYOMO syntax

- Step 7: Solve the model and print results

```
# -----------------------------------
# SOLVE THE MODEL                    |
# -----------------------------------
# Specify the solver engine. Since this is a LP problem, we choose "glpk"
opt = SolverFactory('glpk')

# The following line generates sensitivity tables, but cannot read the
# variable names we have chosen (it creates generic names)
opt.options["ranges"] = 'sens.txt'

results = opt.solve(model)
model.pprint()

# -----------------------------------
# PRINT RESULTS                      |
# -----------------------------------
print("\n --------------- RESULTS --------------- \n");
print("Optimal objective function value ($1000's) =", model.obj());
print("Optimal number of (100's) cases of Cote Ronde to make =", c())
print("Optimal number of (100's) cases of Domaine Paul Autard to make =", p())
```

Manual solution – the “greedy” algorithm

- Can we just make as many P cases as we can, and then make C cases with what’s left?
  - This works for the Domaine Paul Autard problem as formulated with $400 and $600 prices.
  - But it does not always work (Try changing the $600 to $450).
  - The greedy solution is not guaranteed to be optimal.

Sensitivity Analysis

RHS of constraint

Range of LHS where the marginal applies (i.e., before hitting another constraint)

LHS of constraint

```
Problem:
Objective:  obj = 1660 (MAXimum)

   No. Row name     St      Activity         Slack  Lower bound       Activity       Obj coef  Obj value at  Limiting
                                        Marginal  Upper bound          range          range   break point  variable
------ ------------ -- ------------- ------------ ------------  ------------- ------------- ------------- ------------
     1 c_u_GrenacheConstraint_
                    NU       17.00000            .         -Inf       15.00000     -80.00000    1500.00000 c
                                            80.00000     17.00000       22.00000          +Inf    2060.00000 c_u_SyrahConstraint_

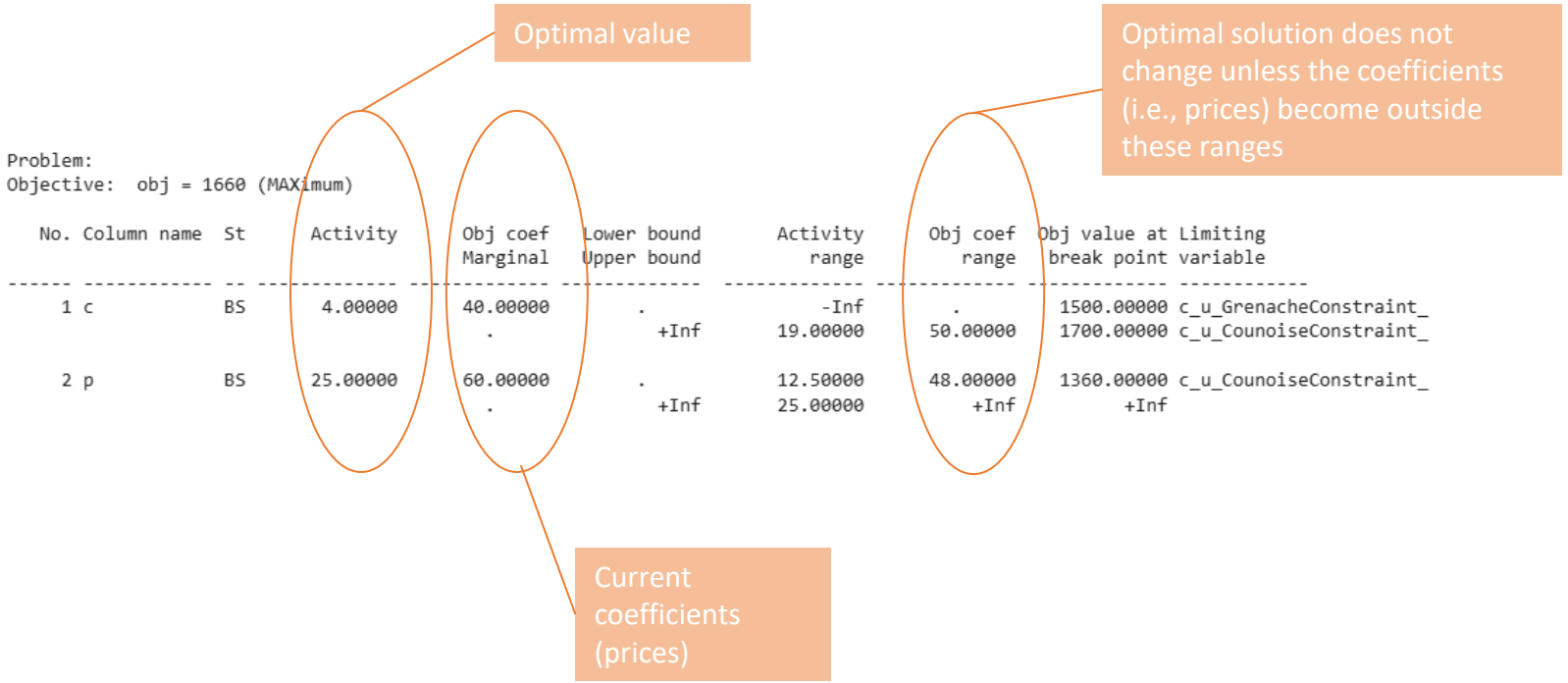
     2 c_u_SyrahConstraint_
                    BS        7.00000         5.00000         -Inf        5.00000     -80.00000    1100.00000 c_u_GrenacheConstraint_
                                                  .         12.00000       17.00000      30.00000    1870.00000 c_u_CounoiseConstraint_

     3 c_u_CounoiseConstraint_
                    NU        5.00000            .         -Inf        2.50000     -60.00000    1510.00000 c_u_SyrahConstraint_
                                            60.00000      5.00000        5.66667          +Inf    1700.00000 c
```

Marginal (Also Shadow price): Increasing the RHS by 1 increases the max of the objective function by 80 within the activity range
This important for understanding the impact of relaxing our constraint. It also indicates a bottleneck.

Slack: We have 5 unused cases. We can lower RHS by 5 and the optimal solution would not change. Not a bottleneck

Sensitivity Analysis



Problem:
Objective: obj = 1660 (MAXimum)

No.	Column name	St	Activity	Obj coef / Marginal	Lower bound / Upper bound	Activity range	Obj coef range	Obj value at break point	Limiting variable
1	c	BS	4.00000	40.00000	.	-Inf	.	1500.00000	c_u_GrenacheConstraint_
				.	+Inf	19.00000	50.00000	1700.00000	c_u_CounoiseConstraint_
2	p	BS	25.00000	60.00000	.	12.50000	48.00000	1360.00000	c_u_CounoiseConstraint_
				.	+Inf	25.00000	+Inf	+Inf	